

Performance Testing

Date	1 st July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Functional Performance Testing, Response Time Testing
Target Module / API	Flask Prediction Module (/predict)
Test Environment	Local System (Windows 11, Python 3.11, Flask Development Server)
Test Date	1 st July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit valid weather parameters for prediction	1	30	Prediction result displayed successfully
2	Multiple consecutive prediction requests	10	60	Application handles requests without failure

3	Invalid or incomplete input values	5	30	Validation message displayed without server crash
4	Continuous prediction requests	20	120	Stable performance with acceptable response time

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Average Response Time	< 2 seconds	0.82 seconds	Pass	Fast prediction response
2	Maximum Response Time	< 5 seconds	1.74 seconds	Pass	Within acceptable limit
3	Throughput	> 10 Requests/sec	18 Requests/sec	Pass	Stable request handling
4	Error Rate	< 1%	0%	Pass	No request failures observed
5	CPU Utilization	< 80%	34%	Pass	Low processor utilization
6	Memory Utilization	< 80%	41%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Section	Details
Key Findings	- The application successfully processed prediction requests with low response time. - The Flask server remained stable during repeated prediction requests. - The Random Forest model generated prediction results quickly after preprocessing. - No application crashes or server errors were observed during testing.
Bottlenecks Identified	- The application currently operates on a local Flask development server, which is suitable for development but not optimized for high concurrent traffic. - Real-time weather API integration has not yet been implemented and is planned as future work.
Optimization Steps Taken	- Optimized feature preprocessing using StandardScaler. - Stored the trained model using Joblib to avoid retraining during runtime. - Implemented efficient request handling within the Flask application. - Organized the application into modular

	components for easier maintenance and scalability.
--	--

ScreenShots:

 **Flood Risk Detected**



Flood Risk Detected

The Machine Learning model predicts a HIGH probability of flooding based on the provided weather parameters.

 **Recommendations**

- ✓ Move to a safer location if necessary.
- ✓ Follow local weather advisories.
- ✓ Avoid flooded roads and low-lying areas.
- ✓ Keep emergency supplies ready.

 Predict Again

 Home

 **No Flood Risk Detected**



No Flood Risk

The Machine Learning model predicts NO significant flood risk based on the provided weather parameters.

 **Recommendations**

- ✓ Weather conditions appear normal.
- ✓ Continue monitoring local forecasts.
- ✓ Stay informed during heavy rainfall seasons.

 Predict Again

 Home